

COLLEGE ERP SYSTEM

Complete Project Overview, Comprehensive File-by-File Registry & Technical Interview Preparation Guide

Document Metadata & Project Scope:

- **Application Type:** Full-Stack College Enterprise Resource Planning (ERP) System
- **Architecture:** Decoupled Single Page Application (SPA) + RESTful Servlet Backend + Relational RDBMS
- **Technology Stack:** React 18, Vite, Context API, Java Servlets (Jakarta EE), Maven, Oracle SQL & PL/SQL
- **User Role Scope:** 7 Role-Based User Portals (Admin, Student, Faculty, HOD, Accounts, Library, Principal)
- **Target Audience:** Software Engineers, System Architects, Technical Evaluators, Viva / Technical Interview Candidates

SECTION 1: Project Overview & System Architecture

1.1 Executive System Overview

The **College ERP System** is a comprehensive, multi-tenant academic management platform designed to automate, streamline, and centralize administrative, academic, financial, and operational tasks for higher education institutions. The system bridges communication and workflow bottlenecks between administrators, department heads, faculty members, students, finance officers, librarians, and executive leadership.

1.2 Core Architectural Principles & Tech Stack

The ERP is engineered using a clean, 3-tier decoupled architecture:

Tier	Technology Stack	Key Responsibilities & Components
Presentation Tier (Frontend)	React 18, Vite, Axios, React Router v6, Lucide React, Pure CSS3 Design System	Single Page Application (SPA) delivering 7 role-based dynamic dashboards, client-side routing, state management via React Context API, token interception via Axios, and responsive modular components.
Application Tier (Backend)	Java 17, Java Servlets (Jakarta EE), Apache Maven, jBCrypt, org.json, CORS Filter	RESTful Web API powered by lightweight Java Servlets handling HTTP requests, BCrypt security hashing, session validation, request filtering (CORS), and business logic routing via DAO pattern.
Data Tier (Database)	Oracle Database 19c/21c, Oracle JDBC (ojdbc8), PL/SQL Stored Procedures, Sequences	Enterprise SQL database managing normalized relational tables (`USERS`, `STUDENTS`, `FEES`, etc.), Oracle sequences for auto-generating IDs, and PL/SQL procedures for complex atomic transactions.

1.3 Role-Based Access Control (RBAC) & 7 User Portals

Security and privacy are enforced at both frontend (Route Guards) and backend (Servlet Filters & DAO checks) levels. The 7 roles and their responsibilities include:

- **1. Administrator (Admin):** System-wide management, creating/editing user accounts, role assignments, system status monitoring, configuration.
- **2. Student:** Personal dashboard, tracking attendance percentages, viewing subject syllabus, downloading/submitting assignments, checking fee status, paying fees online, browsing library books, applying for leaves.
- **3. Faculty / Teacher:** Managing assigned courses, marking student attendance per subject session, creating assignments, evaluating and grading submissions, viewing class timetables.
- **4. Head of Department (HOD):** Overseeing department performance, monitoring faculty & student attendance, approving/rejecting leave applications, assigning subjects to faculty.
- **5. Accounts Officer:** Defining fee structures, recording offline & online payments, issuing payment receipts, tracking overdue fees, generating financial reports.
- **6. Librarian:** Managing book catalog (adding/editing books), processing book issuance and return transactions, calculating overdue fines, maintaining library logs.
- **7. Principal / Director:** Executive dashboard view with college-wide analytics, department performance comparison, broadcast announcements, institutional notices.

SECTION 2: Complete Project File Registry

Below is a detailed inventory and description of all **86 source files** across Database, Backend, and Frontend modules.

File Path	Description & Project Functionality
database/schema.sql	Database DDL script creating normalized Oracle DB tables (`USERS`, `ROLES`, `STUDENTS`, `FACULTY`, `COURSES`, `SUBJECTS`, `ATTENDANCE`, `ASSIGNMENTS`, `SUBMISSIONS`, `FEES`, `FEE_PAYMENTS`, `BOOKS`, `BOOK_TRANSACTIONS`, `LEAVE_REQUESTS`, `NOTICES`) with PK/FK constraints.
database/sequences.sql	Database DDL script defining Oracle sequences (`user_seq`, `course_seq`, `assignment_seq`, etc.) to generate auto-incrementing primary key IDs.
database/procedures.sql	PL/SQL stored procedures defining backend transaction logic (e.g. attendance percentage calculation, fee payment recording, book issuing validation).
database/seed_data.sql	Database DML script populating initial seed data, including default Admin/Faculty/Student accounts, standard course structures, and sample books.
backend/pom.xml	Apache Maven build configuration file defining Java 17 compatibility, WAR packaging, dependencies (`ojdbc8`, `json`, `jbcrypt`, `javax.servlet-api`).
backend/package-lock.json	Npm package lock file for backend auxiliary scripts/tooling.
backend/src/main/webapp/WEB-INF/web.xml	Java EE Web Application Deployment Descriptor mapping Servlets, CORS filters, URL patterns, and session timeouts.
backend/src/main/webapp/index.jsp	Root JSP Page serving as backend HTTP welcome landing test page.
backend/src/main/java/com/collegeerp/config/CORSFilter.java	Servlet Filter implementation intercepting incoming HTTP requests to append CORS headers (`Access-Control-Allow-Origin`, `Methods`, `Headers`).
backend/src/main/java/com/collegeerp/config/DBConnection.java	Singleton JDBC Database Connection factory managing Oracle database connection strings, credentials, and connection instantiation.
backend/src/main/java/com/collegeerp/util/JSONResponse.java	Utility class standardizing HTTP JSON response formatting, success/error payloads, and HTTP status code wrappers.
backend/src/main/java/com/collegeerp/util/PasswordUtil.java	Security utility class wrapping `jbcrypt` for hashing plain-text passwords and verifying hashed credentials.
backend/src/main/java/com/collegeerp/model/User.java	Java POJO domain model representing user entity (id, username, email, passwordHash, role, department, full name).
backend/src/main/java/com/collegeerp/model/Assignment.java	Java POJO domain model representing assignment metadata (id, subjectId, title, description, dueDate, createdBy).
backend/src/main/java/com/collegeerp/model/Attendance.java	Java POJO domain model representing attendance entries (id, studentId, subjectId, date, status, markedBy).
backend/src/main/java/com/collegeerp/model/Book.java	Java POJO domain model representing library books (id, title, author, isbn, category, copies count).
backend/src/main/java/com/collegeerp/model/BookTransaction.java	Java POJO domain model representing book issue/return activity, issue dates, due dates, fine amounts.

backend/src/main/java/com/collegeerp/model/Fee.java	Java POJO domain model representing fee breakdown structures and student financial status.
backend/src/main/java/com/collegeerp/model/FeeRecord.java	Java POJO domain model representing detailed fee billing entries and ledger lines.
backend/src/main/java/com/collegeerp/model/Payment.java	Java POJO domain model representing individual payment transaction receipts and payment mode details.
backend/src/main/java/com/collegeerp/dao/UserDAO.java	Data Access Object executing SQL for user authentication, registration, profile retrieval, user listing, and role updates.
backend/src/main/java/com/collegeerp/dao/AssignmentDAO.java	Data Access Object executing SQL for creating, fetching, submitting, and grading student assignments.
backend/src/main/java/com/collegeerp/dao/AttendanceDAO.java	Data Access Object executing SQL queries for marking student attendance, fetching student logs, and computing percentage stats.
backend/src/main/java/com/collegeerp/dao/FeeDAO.java	Data Access Object executing SQL queries for retrieving fee balances, logging payment transactions, and generating receipts.
backend/src/main/java/com/collegeerp/dao/LibraryDAO.java	Data Access Object executing SQL queries for catalog search, book issue/return status updates, and tracking fines.
backend/src/main/java/com/collegeerp/controller/AuthServlet.java	Java Servlet controller handling <code>/api/auth/*</code> routes (login, register, session verification, logout).
backend/src/main/java/com/collegeerp/controller/AssignmentServlet.java	Java Servlet controller handling <code>/api/assignments/*</code> routes for assignment creation and student submission uploads.
backend/src/main/java/com/collegeerp/controller/AttendanceServlet.java	Java Servlet controller handling <code>/api/attendance/*</code> routes for batch marking and fetching student attendance records.
backend/src/main/java/com/collegeerp/controller/FeeServlet.java	Java Servlet controller handling <code>/api/fee/*</code> routes for fetching fee breakdowns, processing payments, fetching transaction receipts.
backend/src/main/java/com/collegeerp/controller/LibraryServlet.java	Java Servlet controller handling <code>/api/library/*</code> routes for searching catalog, issuing books, returning books, computing fines.
frontend/package.json	Frontend npm project manifest defining dependencies (<code>react</code> , <code>react-router-dom</code> , <code>axios</code> , <code>lucide-react</code>) and build scripts.
frontend/package-lock.json	Frontend npm lock file ensuring deterministic dependency tree resolution.
frontend/vite.config.js	Vite build tool configuration, setting up React plugin, dev server port, proxy paths to Java backend.
frontend/eslint.config.js	ESLint configuration file enforcing Javascript syntax rules and standard React lint rules.
frontend/index.html	Main HTML template for React SPA containing HTML5 container, Google Fonts preconnect, ``.
frontend/.gitignore	Git configuration file specifying build artifacts (<code>dist</code> , <code>node_modules</code>) ignored from source control.
frontend/README.md	Frontend documentation guide detailing setup, execution scripts, and directory layout.
frontend/src/main.jsx	React SPA entry point rendering the root `` component into DOM inside <code>React.StrictMode</code> .
frontend/src/App.jsx	Main application wrapper configuring routing with `` inside global ``.

frontend/src/App.css	Application-level stylesheet defining global layout rules, scrollbar styling, utility classes.
frontend/src/index.css	Base global CSS stylesheet resetting default browser margins, typography settings, box-sizing.
frontend/src/src/assets/index.css	Design system stylesheet specifying Tailwind/custom global CSS variables and base element styles.
frontend/src/src/assets/theme.css	Theme stylesheet defining color palette tokens, dark mode variables, glassmorphism CSS helper classes.
frontend/src/src/context/AuthContext.jsx	React Context provider managing global authentication state (`user`, `token`, `role`), exposing `login()` & `logout()` helpers.
frontend/src/src/routes/AppRoutes.jsx	Main application router mapping path URLs (`/`, `/login`, `/dashboard/*`) to respective view components.
frontend/src/src/routes/ProtectedRoute.jsx	Route guard Higher-Order Component restricting route access based on user authentication status and allowed roles.
frontend/src/src/services/api.js	Centralized Axios client instance configured with base backend API URL, request authorization interceptor, and 401 error handler.
frontend/src/src/services/authService.js	Frontend service module wrapping authentication API calls (`login`, `register`, `getCurrentUser`, `getUsers`).
frontend/src/src/services/assignmentService.js	Frontend service module handling assignment API calls (`getAssignments`, `createAssignment`, `submitAssignment`).
frontend/src/src/services/attendanceService.js	Frontend service module handling attendance API calls (`getAttendance`, `markAttendance`, `getStudentStats`).
frontend/src/src/services/feeService.js	Frontend service module handling fee API calls (`getFeeStatus`, `makePayment`, `getPaymentHistory`).
frontend/src/src/components/common/Datatable.jsx	Reusable dynamic data table component supporting client-side searching, sorting, pagination, and custom column renderers.
frontend/src/src/components/common/Footer.jsx	Global page footer component containing quick links, copyright notice, contact links, institutional branding.
frontend/src/src/components/common/Navbar.jsx	Global header navigation bar rendering user avatar, role badge, quick notifications, mobile drawer toggle.
frontend/src/src/components/common/Sidebar.jsx	Role-aware lateral navigation sidebar dynamically rendering navigation menu items tailored to user's assigned role.
frontend/src/src/components/common/StatCard.jsx	Dashboard stat card widget displaying key performance indicators (KPIs), metrics, icons, and trend indicators.
frontend/src/src/components/modals/ApplyLeaveModal.jsx	Interactive modal dialog allowing students/faculty to compose and submit leave requests with date range and reason.
frontend/src/src/components/modals/IssueBookModal.jsx	Interactive modal dialog enabling librarians to search student accounts and issue cataloged books.
frontend/src/src/components/modals/LogoutModal.jsx	Confirmation modal dialog prompting users to confirm sign-out before clearing session context.
frontend/src/src/components/public/ContactForm.jsx	Public contact form component with client-side validation, submit handler, and success feedback message.

frontend/src/src/components/public/HeroSection.jsx	Landing page hero banner component showcasing call-to-action buttons, tagline, and animated background elements.
frontend/src/src/components/public/ServicesGrid.jsx	Landing page grid component highlighting key ERP features (Attendance, Fees, Library, Analytics).
frontend/src/src/views/public/Home.jsx	Public landing home page aggregating `HeroSection`, `ServicesGrid`, and `ContactForm` components.
frontend/src/src/views/public/About.jsx	Public About Us view detailing college ERP background, mission statement, and platform benefits.
frontend/src/src/views/public/Contact.jsx	Public Contact page rendering contact details, campus address, map view, and inquiry form.
frontend/src/src/views/public/Features.jsx	Public Features showcase page giving prospective users an interactive breakdown of system capabilities.
frontend/src/src/views/auth/Login.jsx	Authentication login view rendering credential form, role selection dropdown, error alert, and submit handlers.
frontend/src/src/views/auth/Register.jsx	Registration view enabling new users to create accounts with input validation and role selection.
frontend/src/src/views/admin/AdminDashboard.jsx	Central management dashboard for System Administrators with user stats, activity logs, quick action shortcuts.
frontend/src/src/views/admin/UserManagementView.jsx	Admin module view for searching, creating, editing, role updating, and suspending system users.
frontend/src/src/views/student/StudentDashboard.jsx	Student portal landing view displaying attendance overview, upcoming deadlines, recent grades, quick links.
frontend/src/src/views/faculty/FacultyDashboard.jsx	Faculty portal landing view displaying assigned classes, pending assignment grading, today's lecture schedule.
frontend/src/src/views/hod/HodDashboard.jsx	Head of Department dashboard displaying department-wide metrics, faculty status, pending leave approvals.
frontend/src/src/views/accounts/AccountsDashboard.jsx	Accounts portal displaying overall fee collections, pending student dues, daily transaction logs.
frontend/src/src/views/library/LibraryDashboard.jsx	Librarian dashboard displaying total books cataloged, active issued books, overdue books count, fines collected.
frontend/src/src/views/principal/PrincipalDashboard.jsx	Executive Principal view featuring institutional high-level KPIs, department comparison graphs, notice board management.
frontend/src/src/shared-modules/AssignmentsView.jsx	Shared view module for browsing assignments, submitting coursework files, viewing evaluated marks and feedback.
frontend/src/src/shared-modules/AttendanceView.jsx	Shared view module for inspecting detailed subject-wise attendance logs, monthly breakdown, percentage progress bars.
frontend/src/src/shared-modules/FeesStatusView.jsx	Shared view module displaying tuition fee itemization, payment history table, and online payment button integration.
frontend/src/src/shared-modules/LeaveRequestsView.jsx	Shared view module for submitting leave applications and viewing approval/rejection status from HOD/Admin.
frontend/src/src/shared-modules/LibraryView.jsx	Shared view module for searching the library book catalog, checking book availability, viewing borrowed books history.

frontend/src/src/shared-modules/NoticesView.jsx	Shared notice board view rendering institutional news, exam announcements, holiday circulars.
frontend/src/src/shared-modules/ProfileView.jsx	Shared user profile view allowing users to view and update personal contact info, profile photo, and password.
frontend/src/src/shared-modules/SubjectsView.jsx	Shared academic module displaying list of registered subjects, assigned course teachers, syllabus details.
frontend/src/src/shared-modules/TimeableView.jsx	Shared timetable module visualizing weekly class schedules, lecture timings, classroom/lab room allocations.
package-lock.json	Root project lockfile ensuring consistent lock states across workspace root.
College_ERP_Project_Layout.pdf	Existing reference PDF document outlining high-level layout of the ERP application.

SECTION 3: Comprehensive Technical Interview Questions & Answers

This section provides **40 in-depth technical interview questions and answers** tailored specifically to this College ERP project stack, architecture, database design, security implementation, and frontend engineering.

3.1 Java Servlets, REST API & Backend Architecture

Q1 [Backend]: Why did you choose lightweight Java Servlets instead of Spring Boot for this ERP backend?

Answer: Choosing raw Java Servlets (Jakarta EE) provides explicit control over the HTTP request/response lifecycle without the implicit overhead, autoconfiguration magic, or heavy startup memory footprint of Spring Boot. It demonstrates a foundational understanding of core Java Enterprise concepts, low-level Servlet filters (`CORSFilter.java`), manual routing, raw JDBC connectivity, and lightweight WAR deployment targeting standard application servers like Apache Tomcat.

Q2 [Backend]: How does the Java Servlet lifecycle handle multi-threaded concurrent user requests in `AuthServlet`?

Answer: When the Servlet container (e.g. Tomcat) starts or receives a request for `AuthServlet`, it instantiates a single instance of the Servlet and invokes its `init()` method. For each incoming client HTTP request, the container allocates a worker thread from its thread pool and calls `service()`, which delegates to `doGet()` or `doPost()`. Because a single Servlet instance handles concurrent threads, instance variables must be avoided to ensure thread safety. Shared resources like database connections are safely checked out per request using stateless DAO objects.

Q3 [Backend]: Explain how CORS (Cross-Origin Resource Sharing) is configured in `CORSFilter.java`.

Answer: Since the React frontend runs on Vite (e.g. `http://localhost:5173`) and the Java backend runs on Tomcat (e.g. `http://localhost:8080`), browsers block cross-origin requests by default due to Same-Origin Policy (SOP). `CORSFilter.java` implements `javax.servlet.Filter` and intercepts every incoming request. It appends HTTP headers: `Access-Control-Allow-Origin: *` (or specific origin), `Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS`, and `Access-Control-Allow-Headers: Content-Type, Authorization`. It also handles pre-flight `OPTIONS` requests by returning an HTTP 200 OK immediately.

Q4 [Backend]: How is user password security implemented in `PasswordUtil.java`?

Answer: Password security utilizes BCrypt hashing via `jBCrypt` library. When a user registers or an admin creates an account, `PasswordUtil.hashPassword(plainText)` generates a salted BCrypt hash with a configurable work factor (cost factor, typically 10-12 iterations). Plain-text passwords are never stored in the database. During login, `PasswordUtil.checkPassword(plainText, storedHash)` verifies the password securely against timing attacks.

Q5 [Backend]: What is the Data Access Object (DAO) pattern used in this project?

Answer: The DAO pattern encapsulates and hides all database interaction details from the Servlet controller layer. For instance, `UserDAO.java` exposes clean Java methods like `authenticateUser(username, password)` and `getUserById(id)` without exposing SQL strings or JDBC `ResultSet` parsing to `AuthServlet.java`. This decouples business logic from persistence logic, making the code testable and maintainable.

Q6 [Backend]: How does DBConnection.java manage database connections, and how could connection pooling improve it?

Answer: `DBConnection.java` provides a centralized factory method `getConnection()` using `DriverManager.getConnection()` with Oracle JDBC driver (`ojdbc8`). While functional, opening and closing physical database TCP connections per request introduces latency. Implementing a Connection Pool (such as HikariCP or Apache Commons DBCP) reuses pre-established database connections from a pool, significantly reducing connection overhead and boosting throughput under high concurrency.

Q7 [Backend]: How do you prevent SQL Injection vulnerabilities in your Java DAO classes?

Answer: All database queries in DAO classes (`UserDAO`, `AttendanceDAO`, `FeeDAO`, etc.) strictly utilize `PreparedStatement` instead of string concatenation. Placeholders (`?`) are used for dynamic parameters, allowing the database driver to compile and sanitize inputs prior to execution, completely neutralizing SQL injection risks.

Q8 [Backend]: How does JSONResponse.java standardize API responses sent back to React?

Answer: `JSONResponse.java` wraps the response writing logic using `org.json.JSONObject`. It sets `response.setContentType("application/json")` and `response.setCharacterEncoding("UTF-8")`. It formats standardized response objects containing `status` (success/error), `message`, and `data` payloads, ensuring the React frontend receives predictable, uniform JSON across all endpoints.

Q9 [Backend]: How are unhandled backend exceptions caught and returned to the client?

Answer: Exceptions inside Servlet methods are wrapped in `try-catch` blocks. When a `SQLException` or `ClassNotFoundException` occurs, it is logged to standard server error logs, and `JSONResponse.sendError(response, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "Internal Server Error")` is invoked. This prevents raw database stack traces from leaking to the frontend.

Q10 [Backend]: Explain session management and authentication verification between Servlets and React.

Answer: Upon successful authentication in `AuthServlet`, a session token or authenticated user payload is generated and returned to the client. The frontend stores this token (in `localStorage` or state) and includes it in the `Authorization` HTTP header for subsequent requests. Servlets check the header or session attributes to grant or deny access to protected endpoints.

3.2 Oracle SQL, Database Schema & PL/SQL

Q11 [Database]: Explain the Entity-Relationship (ER) structure and normalized table design of this ERP system.

Answer: The schema in `schema.sql` is normalized up to 3NF. The central table `USERS` stores login credentials and role references. `STUDENTS` and `FACULTY` maintain 1-to-1 extension relationships with `USERS`. `COURSES` and `SUBJECTS` maintain 1-to-Many relationships with `STUDENTS` and `FACULTY`. Transactional tables like `ATTENDANCE`, `ASSIGNMENTS`, `FEE_PAYMENTS`, and `BOOK_TRANSACTIONS` reference student and faculty primary keys via Foreign Keys with referential integrity constraints.

Q12 [Database]: Why are Oracle Sequences used in sequences.sql instead of identity columns?

Answer: In Oracle Database (especially prior to Oracle 12c or for explicit sequence control), Oracle Sequences like ``user_seq`` generate unique, non-colliding numeric IDs independently of table inserts. Sequences allow pre-fetching next values (``user_seq.NEXTVAL``) within stored procedures or Java code, facilitating complex multi-table inserts within a single transaction.

Q13 [Database]: What PL/SQL Stored Procedures are defined in procedures.sql and what are their advantages?

Answer: ``procedures.sql`` contains compiled PL/SQL stored procedures for complex multi-step operations like attendance percentage compilation, fee receipt ledger updates, and book issuance checks. Executing logic inside Oracle PL/SQL reduces network roundtrips between Java and the Database, speeds up batch execution, and ensures atomic database-level transactions.

Q14 [Database]: How is attendance data stored and queried to compute student attendance percentages?

Answer: Each attendance entry in ``ATTENDANCE`` records ``student_id``, ``subject_id``, ``attendance_date``, and ``status`` ('PRESENT' or 'ABSENT'). To calculate a student's percentage for a subject, SQL queries aggregate count: ``(COUNT(CASE WHEN status='PRESENT' THEN 1 END) * 100.0 / COUNT(*))``. Indexes on ``(student_id, subject_id)`` optimize calculation performance.

Q15 [Database]: How do you prevent race conditions when two students attempt to borrow the last remaining copy of a library book simultaneously?

Answer: In ``LibraryDAO.java`` and PL/SQL procedures, book issuance uses explicit row locking: ``SELECT available_copies FROM BOOKS WHERE book_id = ? FOR UPDATE;``. This locks the target book row until the transaction completes. If ``available_copies > 0``, the copy count is decremented and transaction logged; otherwise, the transaction rolls back, preventing double-issuance.

Q16 [Database]: How are Foreign Key CASCADE constraints configured in schema.sql?

Answer: Foreign key constraints enforce referential integrity. For instance, ``CONSTRAINT fk_student_user FOREIGN KEY (user_id) REFERENCES USERS(user_id) ON DELETE CASCADE`` ensures that if a user account is deleted, associated student profiles or logs are cleanly purged or restricted according to institutional compliance policies.

Q17 [Database]: How would you optimize database query execution if the ATTENDANCE table grows to millions of rows?

Answer: Optimization techniques include: 1) Creating Composite B-Tree Indexes on ``(student_id, subject_id, attendance_date)``, 2) Range Partitioning the ``ATTENDANCE`` table by academic year/semester, 3) Implementing Materialized Views to pre-calculate and cache daily/monthly attendance summary percentages.

Q18 [Database]: What is the distinction between DDL, DML, and DCL files in the /database folder?

Answer: ``schema.sql`` & ``sequences.sql`` are Data Definition Language (DDL) scripts that define database structures (tables, constraints, sequences). ``seed_data.sql`` is Data Manipulation Language (DML) inserting seed records. ``procedures.sql`` contains Data Control and Procedural logic (PL/SQL) for database execution.

Q19 [Database]: How does online fee collection maintain ACID compliance during payment processing?

Answer: Fee payments involve updating ``FEES`` (paid amount & status) and inserting a row into ``FEE_PAYMENTS``. By wrapping both SQL queries within a single transaction (``connection.setAutoCommit(false)``), calling ``connection.commit()`` on success or ``connection.rollback()`` in ``catch``, atomicity and consistency are guaranteed.

Q20 [Database]: What role does seed_data.sql play in development and automated testing?

Answer: `seed_data.sql` populates standardized test data (default Admin credentials, sample Faculty, Students, Subjects, Courses). This enables developers and automated tests to spin up a fully functioning environment instantly without manual data entry.

3.3 React 18, State Management & Modern Frontend Architecture

Q21 [Frontend]: How does AuthContext.jsx manage global authentication state and persistence across browser refreshes?

Answer: `AuthContext.jsx` utilizes React's Context API to expose `user`, `token`, `role`, `login()`, and `logout()` globally. On initial load, an `useEffect` hook reads stored session tokens from `localStorage` or session cookies, validates the session via `authService.getCurrentUser()`, and initializes user state, preventing accidental sign-outs on page refresh.

Q22 [Frontend]: How does ProtectedRoute.jsx enforce Role-Based Access Control on the client side?

Answer: `ProtectedRoute.jsx` wraps protected route components in `AppRoutes.jsx`. It evaluates the authenticated user's role from `AuthContext`. If the user is unauthenticated, it redirects to `/login`. If the user's role is not included in the `allowedRoles` array prop, it redirects to an Unauthorized view or their default dashboard.

Q23 [Frontend]: How is Axios configured in api.js to automatically handle authentication tokens and errors?

Answer: `api.js` creates a centralized Axios instance with `baseUrl`. It attaches a Request Interceptor that automatically injects `Authorization: Bearer` into outgoing request headers. A Response Interceptor catches HTTP `401 Unauthorized` responses and triggers global sign-out and redirection to login.

Q24 [Frontend]: Describe the dashboard component layout hierarchy (Sidebar, Navbar, View components).

Answer: Dashboards (e.g. `StudentDashboard.jsx`) follow a layout wrapper structure: A fixed left `Sidebar.jsx` (dynamically rendering links based on user role), a top `Navbar.jsx` (showing notifications, profile menu, dark mode toggle), and a dynamic main content area rendering specific shared views (`AttendanceView`, `AssignmentsView`).

Q25 [Frontend]: How does DataTable.jsx achieve reusable client-side searching, sorting, and pagination?

Answer: `DataTable.jsx` accepts `data` array and `columns` configuration props. It uses internal React state for search filter term, current page index, and sort order. A `useMemo` hook filters and sorts data efficiently without triggering unnecessary re-renders.

Q26 [Frontend]: Explain the directory separation between /views, /components, /services, and /shared-modules.

Answer: `/views` contains page-level route targets (e.g. Dashboards, Login). `/components` contains UI widgets (`Navbar`, `DataTable`, `Modals`). `/services` abstracts API HTTP calls (`authService.js`). `/shared-modules` contains domain-specific feature views (`AttendanceView`, `LibraryView`) rendered across multiple role dashboards.

Q27 [Frontend]: What are the benefits of decoupling API logic into service files (e.g. assignmentService.js)?

Answer: Decoupling API calls keeps React components pure and focused on UI rendering and local state management. If an API endpoint URL or payload structure changes, modifications are confined to the service file without touching component code.

Q28 [Frontend]: How does React Router v6 handle declarative client-side navigation in AppRoutes.jsx?

Answer: `AppRoutes.jsx` uses ``, ``, and `` components. It defines public routes (`/`, `/about`, `/login`) and protected nested dashboard routes, delivering seamless SPA navigation without full page reloads.

Q29 [Frontend]: How does the frontend handle loading states and API error feedback?

Answer: Components maintain `loading` (boolean) and `error` (string) state variables. During async service calls, spinners are displayed. If an API call fails, error messages are rendered in alert boxes or toast notifications.

Q30 [Frontend]: How is styling structured in theme.css and index.css?

Answer: `theme.css` specifies CSS custom properties (variables) for primary colors, dark mode surfaces, text colors, and glassmorphic backdrop filters. `index.css` sets global font resets and layout utilities, delivering a cohesive visual theme.

3.4 System Design, Security, Scalability & Testing

Q31 [System Design]: If 50,000 students log in simultaneously during exam result releases, how would you scale this ERP?

Answer: Scaling strategy: 1) Deploy multiple Java Servlet instances behind a Load Balancer (Nginx / AWS ALB) with stateless sessions, 2) Introduce Redis Cache to serve static course catalog and student profile data, 3) Implement Oracle DB read-replicas for read-heavy queries, 4) Enable CDN caching for React static frontend assets.

Q32 [System Design]: How would you refactor this Java Servlets monolith into a Spring Boot Microservices architecture?

Answer: Refactoring steps: 1) Decompose into microservices domain boundaries (Auth Service, Student Service, Attendance Service, Fee Service, Library Service), 2) Implement an API Gateway (Spring Cloud Gateway) for routing and authentication, 3) Use RabbitMQ / Kafka for asynchronous messaging (e.g. fee payment triggers notification service).

Q33 [Security]: Compare Session-based authentication vs JWT (JSON Web Tokens) for this College ERP.

Answer: Session-based auth stores session state on the server (requiring sticky sessions or distributed session store). JWT is stateless: the server signs a cryptographically secure token containing user ID and role, which the client sends in HTTP headers. JWT was chosen for stateless SPA scalability.

Q34 [Security]: How would you securely implement file upload functionality for student assignment submissions?

Answer: Security measures: 1) Restrict allowed file MIME types (PDF, DOCX) and enforce max file size limits (10MB), 2) Sanitize file names to prevent directory traversal (`../`), 3) Store files outside the web root or in Cloud Object Storage (AWS S3) with generated UUIDs, 4) Perform virus scanning before saving.

Q35 [Security]: How is data privacy enforced so students cannot view other students' grades or fee statements?

Answer: Backend Authorization: Servlet endpoints extract the user ID and role directly from the authenticated token/session, ignoring client-supplied student IDs in URL parameters unless the user holds Admin/Faculty privileges. Queries strictly append `WHERE student\_id = :authenticated\_user\_id`.

Q36 [System Design]: What caching mechanism would you introduce to reduce database load for static notices?

Answer: Implement Redis in-memory cache. When `NoticesView` requests active notices, the backend checks Redis (`notices:active`). If present (cache hit), it returns immediately. If missing (cache miss), it queries Oracle DB, writes result to Redis with a 15-minute TTL, and returns payload.

Q37 [System Design]: How would you implement real-time notifications for grade publications and announcements?

Answer: Integrate WebSockets (using HTML5 WebSocket API on frontend and `@ServerEndpoint` / Spring WebSocket on backend). When a faculty publishes grades, the server pushes a real-time notification payload directly to the connected student's socket channel.

Q38 [Operations]: What database backup and disaster recovery strategies should be implemented for this ERP?

Answer: Strategies: 1) Oracle RMAN (Recovery Manager) for daily automated full backups and incremental hourly archive log backups, 2) Multi-region Database Replication (Oracle Data Guard) for automated failover during cloud region outages.

Q39 [Security]: What audit logging mechanisms should be implemented for sensitive financial transactions?

Answer: Maintain an immutable `AUDIT\_LOGS` table recording `log\_id`, `actor\_id`, `action\_type` ('FEE_WAIVER', 'USER_DELETE'), `target\_id`, `timestamp`, `ip\_address`, and `old\_value/new\_value` JSON diffs. Database triggers enforce that audit logs cannot be modified or deleted.

Q40 [Testing]: What automated testing strategy would you recommend for this full-stack project?

Answer: Testing Strategy: 1) **Unit Testing:** JUnit 5 and Mockito for testing Java DAO and Servlet helper logic; Jest/React Testing Library for React components, 2) **Integration Testing:** Testcontainers to spin up temporary Oracle DB containers for DAO SQL verification, 3) **End-to-End (E2E) Testing:** Cypress or Playwright automating complete user workflows (Login -> Mark Attendance -> Pay Fee).

End of Document — College ERP System Architectural Specification, File Registry & Technical Interview Guide.